

COMPUTER AND NETWORK SECURITY
PROJECT REPORT

Malicious URL Detection System

Browser-Based Threat Sandbox

Student	Ouadah Lina Selma
Student	Meriche Yasmine
Lab Instructor	Dr. Sahraoui Yacine
Lec. Instructor	Dr. Karim Lounis
Course	Network & System Security
Date	April 2026

0. Contents

1	Problem	2
2	Solution	2
3	Implementation	2
3.1	Evidence Collection — Ouadah Lina Selma	2
3.2	Threat Analysis — Meriche Yasmine	3
3.3	Infrastructure	3
4	How to Use the Website	4
4.1	Step 1 — Enter a URL	4
4.2	Step 2 — Read the Results	4
4.3	Step 3 — Analyze Another URL	8
5	Results	8
6	Discussion	8
7	Conclusion	8

1. Problem

You receive a suspicious link. How do you know if it is safe before clicking it?

Static tools fail here — they only recognise threats they have seen before. A brand-new phishing page, an obfuscated JavaScript dropper, or a redirect chain to malware will sail right through them.

Our answer: open the link in a controlled browser, watch every single thing it does, and score it.

2. Solution

A two-part pipeline connected through a shared `evidence.json` file:

URL Input → Browser (Playwright) → evidence.json → Analysis Pipeline →
Verdict

Evidence Collection (Ouadah Lina Selma) visits the URL in a real headless Chromium browser, instruments it to catch JavaScript tricks, logs every network request, verifies TLS, and records any file downloads.

Threat Analysis (Meriche Yasmine) reads the evidence, queries three threat intelligence APIs, applies 16 heuristic rules, and produces a risk score from 0 to 100.

Everything runs inside a Docker container — every scan spins up a fresh isolated container using the `malicious-url-scanner` image, runs the full pipeline inside it, then destroys the container when done. This means the host machine is never exposed to whatever the scanned page does, even if it tries to download or execute malware.

3. Implementation

The project is made of 12 files split across two pipelines. Below is a brief explanation of what each file does.

3.1. Evidence Collection — Ouadah Lina Selma

browser.py The core of Pipeline 1. It launches a headless Chromium browser using Playwright, injects the JavaScript spy script before the page loads, then navigates to the target URL. It captures every HTTP request, every popup, every file download, and takes a full-page screenshot. All collected data is returned as a Python dictionary.

js.hooks.py Generates the JavaScript spy script as a string. The script replaces four browser functions with monitored versions: `eval()` (catches dynamically executed code), `document.write()` (catches injected HTML), `alert/confirm/prompt` (catches social engineering popups), and the form submit event (catches credential harvesting).

```
1 const _re = window.eval;
2 window.eval = function (code) {
3     window.__spy.eval_calls.push(String(code));
4     return _re(code);
5 };
```

Listing 1: `eval()` hook — catching obfuscated JS at runtime

network_capture.py Handles TLS certificate verification by performing an independent SSL handshake using Python’s `ssl` module. Extracts whether the certificate is valid and who issued it. Also provides helper functions for logging HTTP requests and reading cookies from the browser context.

evidence_builder.py Orchestrates Pipeline 1. Calls **browser.py** to visit the URL, merges the TLS info, and assembles everything into a single structured **evidence.json** file. This file is the interface between Pipeline 1 and Pipeline 2.

3.2. Threat Analysis — Meriche Yasmine

ioc_extractor.py Reads **evidence.json** and extracts Indicators of Compromise (IOCs): unique IP addresses, domain names, full URLs, file hashes (MD5/SHA256), TLS status, cookies, and page title. These are passed to the intelligence and heuristics steps.

threat_intel.py Queries three external reputation APIs for every scan:

Service	What it checks	Max score added
VirusTotal	Domain against 90+ antivirus engines	+40
AbuseIPDB	IP abuse confidence score (0–100)	+30
URLhaus	URL against known malware database	+30

heuristics.py Applies 16 rule-based checks to the raw evidence. Each rule inspects a specific aspect of the scan data and returns a description if triggered. The most impactful rules:

Rule	Weight
Suspicious file download	35
Data exfiltration (POST out)	35
Obfuscated JavaScript	25
Credential harvesting form	25
Misleading page title	25
Crypto mining script	25
Invalid TLS certificate	20
Typosquatting domain	20
Insecure cookies	10

scorer.py Adds up the weights of all triggered heuristic rules and all threat intelligence contributions. The total is capped at 100. Returns the final score with a breakdown of how much came from intel vs. heuristics.

verdict.py Maps the score to a verdict label and writes **verdict.json**. Also orchestrates the entire Pipeline 2 sequence: extract IOCs → query intel → run heuristics → score → save verdict.

CLEAN	SUSPICIOUS	MALICIOUS
0 – 30	31 – 69	70 – 100

3.3. Infrastructure

main.py The entry point of the full pipeline. Called with a URL as argument. Calls **evidence_builder.py** first, then **verdict.py**, and prints the final JSON result to stdout so **app.py** can read it.

sandbox.py Wraps the entire scan inside a Docker container using the Python docker SDK. Creates a fresh **malicious-url-scanner** container for each scan, mounts the project folder read-only and the **output/** folder read-write, waits up to 120 seconds, then destroys the container. The host machine is never exposed to the scanned page.

app.py The Flask web server. Exposes three endpoints: **GET /** serves the frontend, **POST /scan** triggers **sandbox.py** and returns the verdict and evidence as JSON, and **GET /screenshot** serves the screenshot taken during the scan.

page.html The single-page frontend. Three views: home (URL input), analyzing (animated progress while Docker runs), and results (verdict ring, score, signals, technical details accordion, and page screenshot). Communicates with the backend via **fetch()** calls to **/scan** and **/screenshot**.

4. How to Use the Website

4.1. Step 1 — Enter a URL

Open the browser and go to **http://127.0.0.1:5000**. You will see the home page with a single input field. Type or paste any URL you want to check and click **Scan URL**. The interface immediately begins scanning the URL in the isolated sandbox environment, as shown below with a live scan of a Netflix phishing clone.



Figure 1: Home page — scanning **https://brunovcg.github.io/Netflix-login-page-clone**. The status bar confirms the scan is in progress and may take up to 60 seconds.

4.2. Step 2 — Read the Results

When the scan finishes, the result page appears with:

- **Verdict ring** — a circular arc filled proportionally to the risk score, colored green (clean), orange (suspicious), or red (malicious)
- **Risk score** — a number from 0 to 100
- **Triggered signals** — the exact reasons the score was raised
- **Technical details** — redirect count, request count, TLS info, cookies, scan duration (click to expand)

- **Page screenshot** — a visual capture of what the scanned page looked like inside the sandbox

Figure 2 shows the result for the Netflix phishing clone, which received a **SUSPICIOUS** — **55/100** verdict. Two signals were triggered: VirusTotal flagged the domain `brunovcg.github.io` across 16 engines as malicious, and AbuseIPDB rated the hosting IP (185.199.109.153) as suspicious with a score of 32/100. The scan recorded 9 HTTP requests, 2 redirects, and a 14.72-second scan duration. TLS was present with a valid Let's Encrypt certificate.

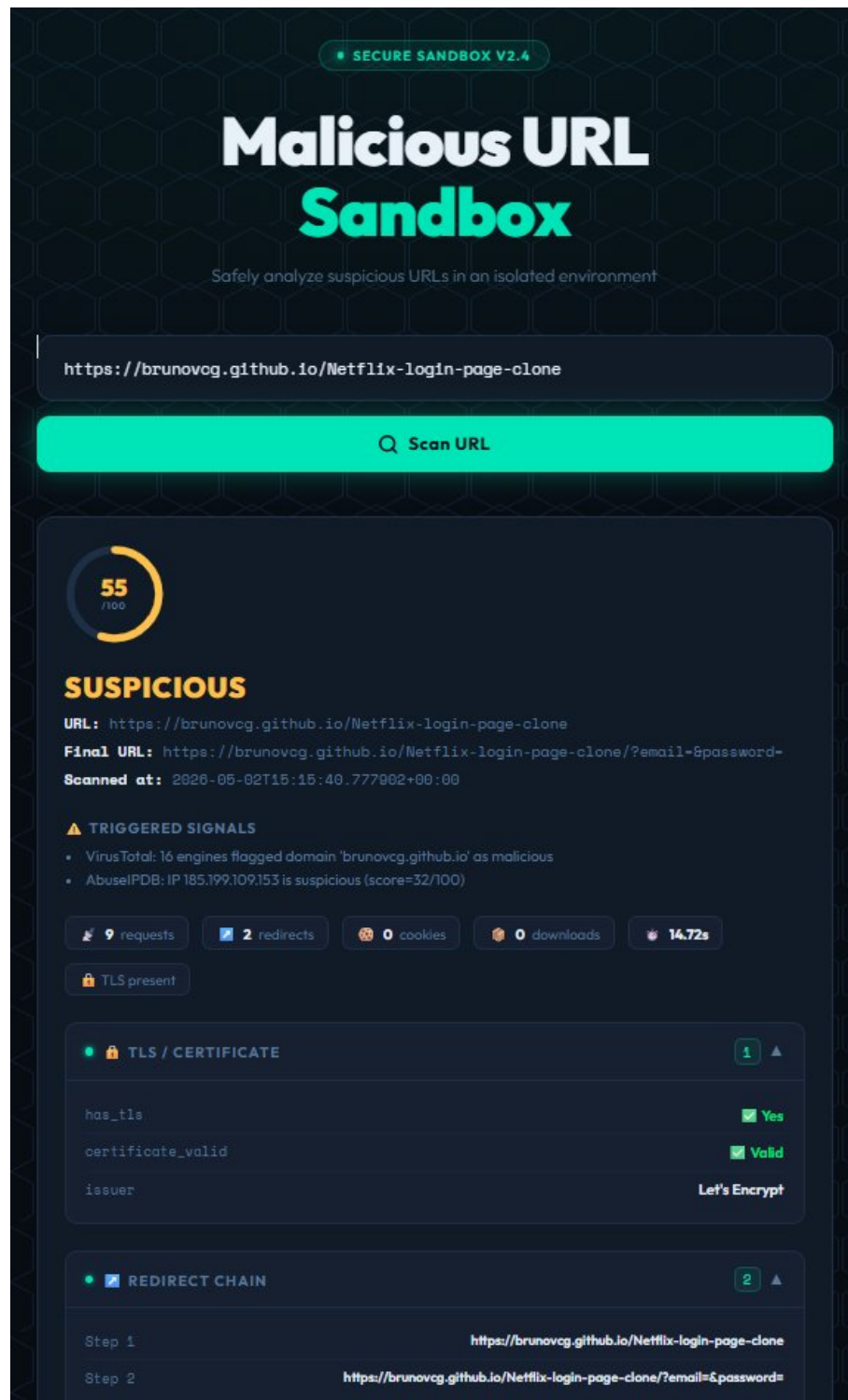


Figure 2: Result page — **SUSPICIOUS 55/100** verdict for the Netflix login page clone. Triggered signals include VirusTotal (16 engines) and AbuseIPDB (score 32/100). The redirect chain (2 steps) and TLS details are visible in the accordion panels below.

Figure 3 shows the sandbox page screenshot captured during the scan, confirming the page renders a convincing Netflix login interface designed to harvest credentials.

The image displays a dark-themed interface for a sandbox analysis tool. It contains several sections:

- EVAL() CALLS (1)**: A code block showing a JavaScript function call: `() => { window.__spy_reading = true; const data = { eval_calls: window.__spy.eval_calls, dom_writes: window.__spy.dom_writes, popups: window.__spy.popups, form_submissions: window.__spy.form_submissions, }; window.__spy_reading = false; return data; }`
- DOCUMENT.WRITE() CALLS (0)**: None
- POPUPS / ALERTS (0)**: None
- FORM SUBMISSIONS (0)**: None
- COOKIES**: 0
- DOWNLOADED FILES**: 0
- PAGE SCREENSHOT**: 1 (with an upward arrow). The screenshot shows a Netflix login page with a grid of movie posters in the background and a central login form with fields for email and password, and a red 'Entrar' button.
- SCAN METADATA**: 1 (with an upward arrow). A table of scan results:

Field	Value
sandbox_id	sandbox-a09905bf
scan_timestamp	2026-05-02T15:15:40.777902+00:00
exited_cleanly	Yes
page_title	Netflix
score_breakdown	Threat Intel: 55 Heuristics: 0

Figure 3: Sandbox page screenshot and scan metadata for the Netflix phishing clone. The page title resolves to *Netflix* and the score breakdown shows Threat Intel: 55, Heuristics: 0. The `eval()` hook captured one call, and zero form submissions were recorded (the form was not submitted during scanning).

4.3. Step 3 — Analyze Another URL

Click **Analyze Another URL** at the bottom of the result page to return to the home screen and run a new scan.

5. Results

The system was tested live on <https://brunovcg.github.io/Netflix-login-page-clone>. The scan finished in **14.72 seconds** and returned the following:

Metric	Value
Final URL	.../Netflix-login-page-clone?email=&password=
Redirects	2
HTTP requests	9
Cookies	0
Downloads	0
TLS issuer	Let's Encrypt
Scan duration	14.72 s

Two signals triggered:

- **VirusTotal:** domain `brunovcg.github.io` flagged by 16 antivirus engines as malicious.
- **AbuseIPDB:** IP 185.199.109.153 is suspicious (score 32/100).

The final URL reveals the redirect appends `?email=&password=` parameters, a strong indicator of credential harvesting behavior. The page screenshot (Figure 3) confirms a pixel-perfect Netflix login clone. The `eval()` hook fired once, suggesting some dynamic JavaScript execution, while no form submissions were recorded during the automated scan.

6. Discussion

Running a real browser caught things a static scanner would miss entirely: the redirect chain appending credential parameters, the IP reputation, and the visual clone of a legitimate login page. Docker isolation kept the host machine safe throughout. The two-pipeline split let both team members work in parallel without blocking each other.

Limitations identified:

- Pages that detect headless browsers may serve blank or benign content (evasion).
- Free API tiers have rate limits — heavy use exhausts them quickly.
- No anti-evasion techniques (mouse simulation, random delays) have been implemented yet.

7. Conclusion

We built a working sandbox that visits a URL, watches its behavior, cross-checks it against three threat databases, and scores it — all inside Docker, in under 30 seconds. The live test on a Netflix phishing clone returned a correct **SUSPICIOUS** — **55/100** verdict, picking up real signals: 16 antivirus engines flagging the domain, a suspicious hosting IP, and a redirect chain that appends credential parameters to the final URL.

The system is modular — heuristics, scoring thresholds, and intel sources can all be swapped out independently as the threat landscape changes.

— *End of report* —